JML Identity Lifecycle Engine

Joiner Module

Identity Governance Architecture

Microsoft Entra ID | Azure Functions | Microsoft Graph API

Henry Ezihe

Identity and Access Management Engineer

May 2026

Table of Contents

1. Executive Summary
2. Business Context
3. Identity Governance Problem Statement
4. Architectural Design Principles
5. Architectural Objectives
6. High-Level Architecture
7. Joiner Workflow Lifecycle
8. Governance Controls Embedded Into Provisioning
9. Engineering Decisions
10. Technology Stack
11. Validation and Testing
12. Operational Boundaries
13. Project Roadmap
14. Architecture Summary

1. Executive Summary

This document covers the Joiner module of the JML Identity Lifecycle Engine: a policy-driven provisioning pipeline built on Microsoft Entra ID, Microsoft Graph API, and Azure Functions. The Joiner module handles new employee and contractor onboarding into an enterprise cloud identity environment.

Manual onboarding carries predictable failure modes. Group memberships are assigned inconsistently, governance checks are absent or run after provisioning has already completed, and there is often no structured record of what was granted or why. The downstream effects are access inconsistency, compliance gaps, and remediation work that could have been avoided.

The Joiner module runs governance validation before creating any identity object, resolves entitlements from a versioned policy rule set, and produces a structured decision record for every event. Access is assigned once, correctly, with a complete evidence trail.

2. Business Context

The State of Enterprise Identity Onboarding

When a new employee joins, IT must create an account, assign the right groups, configure application access, and in some roles set up elevated permissions. Done well, this takes a short time and produces a clean, auditable record. Done through manual ticketing or ad hoc scripts, it introduces inconsistency from the first day.

The consequences are not theoretical. People with the same job title in different departments end up with different access. Former employees retain access after their last day because offboarding was not triggered. Auditors ask for the policy behind a specific group assignment and the answer is reconstruction, not documentation.

Operational Problems

- Inconsistent access provisioning: group assignments vary between individuals in the same role depending on who processed the request.
- Delayed access: manual handoffs between HR and IT slow down onboarding and reduce productivity in the first week.
- Excessive permissions: without a defined entitlement model, access is added to cover future needs rather than assigned to meet current requirements.
- Manual HR-to-IT handoffs: provisioning depends on HR data reaching IT via email, spreadsheet, or informal communication. Data quality and timing are not guaranteed.

- Orphaned accounts: identities remain active after employment ends when offboarding is not automated or monitored.
- Audit gaps: there is no structured record of what access was granted, what justified it, or what the approval chain was. Evidence is assembled from logs and tickets after the fact.
- Role inconsistencies: the same policy is applied differently across teams, creating an access landscape that becomes harder to interpret over time.
- Access sprawl: legitimate access and historical accumulated access become indistinguishable without a clear entitlement model.

Business Impact

These problems do not stay operational. Delayed access costs productivity. Excessive permissions create insider risk. Audit gaps create compliance exposure during certification cycles and regulatory reviews. For organizations in regulated sectors, the absence of a structured, documented provisioning process is a control gap, not just an efficiency problem.

3. Identity Governance Problem Statement

The core problem in identity provisioning is enforcing access policy consistently, at the point of provisioning, across every event. Entitlements must be derived from a defined policy, evaluated before they are applied, and recorded in a way that can be examined independently. Most provisioning approaches are weak on at least two of those three.

The table below sets out the specific risk categories the Joiner module was built to address.

Risk	Description	Business Impact
Duplicate provisioning	The same onboarding event processed more than once	Duplicate accounts and inconsistent directory state
Excessive entitlements	Role mapping assigns group membership beyond what the role requires	Least-privilege violation and compliance exposure
Provisioning drift	Directory state diverges from defined access policy over time	Gaps surfaced only during periodic access reviews
Race conditions	Multiple processes modify the same identity record concurrently	Conflicting access state with no reliable recovery path
Stale HR data	An unresolved department or job title drives group assignment logic	Incorrect entitlement resolution and unauthorized access
Manual onboarding	IT personnel assign groups without a policy reference	Human error, inconsistency, and no structured evidence

These risks accumulate. A modest error rate across a large provisioning volume produces an access state that is difficult to audit and expensive to remediate. Adding more process controls to a manual workflow does not solve this. The architecture removes the conditions that make these risks possible.

4. Architectural Design Principles

The following principles shaped the structural decisions in the Joiner module. They are stated here because they constrain implementation choices in ways that are not always obvious from reading the architecture alone.

Principle	Architectural Meaning
Fail Closed	When governance validation fails or input data is incomplete, provisioning stops. The system does not proceed on assumptions.
Separation of Concerns	Policy evaluation, provisioning execution, validation, and audit recording are separate components. No layer crosses into the responsibility of another.
Deterministic Processing	The same lifecycle event produces the same entitlement outcome on every run. Runtime state does not influence output.
Explicit State Management	Every identity event exists in a named lifecycle state. Transitions are enforced and recorded. No event exists outside the state model.
Governance Before Access	Policy evaluation must complete before any identity object is created. There is no provisional provisioning followed by correction.
Observable Operations	Every provisioning action produces a structured, traceable record designed for audit, not just for operational logging.

Each principle has a direct expression in the system. Fail closed appears in the governance gates that stop provisioning when validation fails. Separation of concerns appears in the distinct layers for policy evaluation, provisioning execution, and audit recording. Observable operations appear in the decision report written for every event, regardless of outcome.

5. Architectural Objectives

These objectives were defined before build began and treated as constraints throughout. They translate the design principles from Section 4 into specific, testable properties.

Objective	What It Means in Practice
Policy-Driven Provisioning	Every group assignment and RBAC entitlement is resolved from an externally configurable rule set. Nothing is hardcoded.
Deterministic Identity State	The same HR input produces the same provisioning outcome. Results are repeatable and predictable across runs.
Idempotent Event Processing	Processing the same event more than once produces one outcome. Retries and duplicate submissions do not require manual cleanup.
Governance-First Onboarding	No Entra ID object is created until the canonical payload has cleared a pre-provision governance gate.
Least-Privilege Access Assignment	Entitlements reflect the minimum access required for the role and employment type. No speculative or inherited access is assigned.
Decoupled Entitlement Resolution	Policy evaluation is separate from provisioning. Changing an access rule does not require redeployment of the engine.
Complete Auditability	Every provisioning decision is recorded in a structured JSON report at the time of the event. Every rule that contributed to the outcome is identified.
Scalable Policy Management	The rule set and lookup tables are version-controlled files in Azure Storage. Policy can be updated independently of the codebase.

Where a simpler approach would have compromised one of these objectives, the more rigorous option was taken. The engineering decisions in Section 9 each trace back to one or more of these.

6. High-Level Architecture

The pipeline is linear. No record advances to the next stage without satisfying the requirements of the current one. Every failure path has a defined handling route and produces a record the operator can act on.

Each component has one responsibility. The provisioning layer holds no policy logic. The policy engine makes no Graph API calls. The audit layer makes no provisioning decisions. This boundary discipline is what allows each component to be tested and changed in isolation.

Component	Purpose
HR Input Layer	Ingests identity events from the HR system via CSV or live API feed. The CSV schema matches the API response shape, so switching to a real-time source requires no changes to downstream components.

Component	Purpose
Normalization Layer	Resolves raw HR field values to canonical equivalents before downstream components see them. Records with unresolvable values are held, not passed forward.
Event Store	Assigns a deterministic SHA-256 event identifier before provisioning begins. Protects against duplicate processing on retry, replay, or concurrent execution.
Entitlement Resolution Layer	Evaluates named policy rule objects against the canonical identity record to determine which security groups the identity should receive. Multiple rules can contribute to the same identity.
Pre-Provision Validation Gate	Evaluates the canonical payload against 27 governance rules before any Entra ID object is created. A failing record is held with structured reason codes. Provisioning cannot proceed without a passing result.
Graph API Provisioning Layer	Creates the Entra ID user and assigns resolved security group memberships via Graph API. RBAC entitlements are delivered through group membership, not direct role assignment. All calls are safe to repeat.
PIM Eligibility Layer	Assigns group-based Privileged Identity Management eligibility for identities requiring just-in-time privileged access. Requires Entra ID P2.
Post-Provision Validation Gate	After provisioning, the governance engine re-evaluates the actual Entra ID object and confirms directory state matches the entitlement set. Discrepancies are recorded.
Hold Queue	Holds identities that fail any gate as state machine records with reason codes, retry counts, and a supervised release path. No record is discarded silently.
Audit Layer	Writes a structured JSON decision report for every identity event regardless of outcome. One report per event, produced at time of processing.

The governance validation engine runs as a separately deployed PowerShell Azure Function. The Python provisioning pipeline calls it over HTTP and receives a structured response. Keeping them separate means the governance engine can be updated, tested, and deployed independently of the provisioning runtime.

7. Joiner Workflow Lifecycle

Each stage below describes what happens, what it checks, and where a failing record goes. No stage is optional.

Stage 01 HR Data Ingestion and Action Derivation

An HR record enters the pipeline from the source system, either as a CSV file or via a direct HR API connection. The CSV schema matches the shape of the API response, so replacing CSV with a live API feed requires no changes to downstream stages. Each record is checked for structural completeness: required fields must be present and non-empty. Records that fail this check are sent to the hold queue with a rejection reason before they go any further.

For records ingested through the HR API, the action deriver compares each record against live Entra ID state. It determines whether the record is a new Joiner, an existing identity that has changed role, or a record with no actionable difference from the current directory state. Only records that require action enter the provisioning pipeline. The rest are filtered out at this stage.

Stage 02 Canonical Normalization

Raw HR field values are resolved to canonical equivalents using a lookup table stored in Azure Storage. A department value of 'sales dept', 'SALES', or 'Sales Dept' all resolve to 'Sales'. A job title of 'SM' or 'sales mgr.' resolves to 'Sales Manager'. Records with values the lookup table cannot resolve are sent to the hold queue. The lookup table is a file in Azure Storage and can be updated without redeploying the engine.

Stage 03 Event Store Claim

Before any provisioning work starts, the engine assigns a SHA-256 event identifier derived from the employee ID, the action type, and the start date. It writes this identifier to Azure Table Storage. If the identifier already exists, the engine exits without doing anything further. This is how the pipeline handles retries, duplicate submissions, and concurrent function invocations without producing duplicate provisioning outcomes.

Stage 04 Entitlement Resolution

The canonical identity record is evaluated against a set of named policy rule objects stored in Azure Storage. Each rule defines conditions on job title, department, and employment type, and specifies which security groups a matching identity should receive. More than one rule can contribute groups to the same identity. The result is the definitive list of group memberships for this identity. Each entitlement is tagged with the rule identifier that produced it, which is written to the decision report.

Stage 05 Pre-Provision Governance Gate

The entitlement set and identity payload are submitted to the governance validation engine, which evaluates them against 27 rules. No Entra ID object has been created at this point and no Graph API calls are made during this evaluation. The rules check employment type constraints, manager association, duplicate UPN conflicts, and privilege tier classification. A Contractor payload assigned to a management-tier group is rejected here before any directory write occurs. A failing evaluation sends the event to the hold queue with specific rule identifiers and failure reasons. Provisioning does not proceed without a passing result.

Stage 06 Graph API Provisioning

With the governance gate cleared, the engine creates the Entra ID user object and assigns the resolved group memberships via Microsoft Graph API. RBAC entitlements are not assigned directly to the user. They are delivered through group membership: the groups carry the RBAC roles, and assigning the user to the group delivers the entitlement. This means RBAC policy changes apply at the group level and take effect without modifying individual user objects. All Graph API calls are safe to repeat. Each action is written to the decision report as it completes, so a partial failure produces a precise record of what succeeded before the failure occurred.

Stage 07 PIM Eligibility Assignment

For identities whose entitlement set includes PIM eligibility, the engine submits group-based eligibility schedule requests. This enables just-in-time access to privileged groups without persistent role assignment. PIM eligibility requires Entra ID P2. If the license is absent, the engine records a warning in the decision report and continues. The rest of provisioning is not affected.

Stage 08 Post-Provision Governance Verification

After provisioning completes, the governance engine fetches the actual Entra ID object using three targeted Graph API calls: one for the user record, one for the user's group memberships, and one per group to resolve display names. It then confirms that the real directory state matches the expected entitlement set. Discrepancies are recorded in the decision report. Hygiene checks that cannot be meaningful at the moment of account creation, such as MFA registration, are recorded as informational warnings rather than failures.

Stage 09 Decision Report

A structured JSON report is written for every identity event regardless of outcome. It records the identity, the event type, the normalization result, every action taken, every rule that contributed to the outcome, both validation gate results, and the full reason chain for any event that did not complete. This report is the compliance record for the event. It is written at the time of processing and is not modified afterwards.

8. Governance Controls Embedded Into Provisioning

The controls listed below are structural properties of the architecture. They run at runtime regardless of operator behavior and cannot be bypassed through operational shortcuts.

Control	Description
Pre-Provision Validation Gate	A 27-rule governance evaluation runs against the canonical payload before any Entra ID object is created. Failing records are held with structured reason codes. Provisioning cannot proceed without a passing result.
Deterministic Event Identifiers	Each event receives a SHA-256 hash derived from the employee ID, action type, and start date. The same input always produces the same identifier, protecting against duplicate processing across retry and concurrent execution.
Concurrency Locking	A processing lock is written to the event store at the start of each run and expires automatically after ten minutes. Two function instances cannot process the same event simultaneously.
Idempotent Graph API Operations	All Graph API operations are safe to repeat. Creating a user that exists, or adding a membership that already exists, produces no unintended result.

Control	Description
Hold Queue State Machine	Records that fail normalization or governance evaluation enter a state machine with defined transitions, reason codes, and a supervised release path. Invalid transitions are rejected at runtime.
Employment Type Enforcement	Employment type is evaluated at the payload level before provisioning. Contractors and Interns cannot be placed in management-tier or privileged groups. This check runs before any directory write.
Structured Decision Records	A JSON report is written for every event at the time of processing. It captures every action taken, every rule that fired, and every failure reason. Evidence is not reconstructed from logs retrospectively.
Post-Provision Verification	After provisioning, the governance engine fetches the real Entra ID object with three targeted Graph calls and confirms actual directory state matches the expected entitlement set.
FIFO Conflict Queue	A new event arriving for an identity with an active event in progress is queued in arrival order. If the preceding event failed, the queue waits for human review before the next event runs.

Structural enforcement matters because it does not depend on people following a procedure. An operator cannot skip the governance gate under time pressure. A function retry cannot create a duplicate account. A Contractor payload cannot clear the employment type check into a management-tier group. These outcomes come from how the system is built, not from how it is operated.

9. Engineering Decisions

The decisions below each had a direct effect on how the system behaves. Each one is traceable to a specific objective in Section 5 or principle in Section 4.

Decision	Rationale	Outcome
Policy as external JSON rule objects	Access policy changes are file edits, not code changes. Governance teams can update the rule set without engineering involvement and without redeploying the engine.	Policy and code lifecycle decoupled
SHA-256 deterministic event identifiers	The same HR payload always produces the same event ID. Azure Table Storage rejects a duplicate insert atomically. Retries and concurrent instances resolve safely without coordination overhead.	Single-outcome processing on retry and replay

Decision	Rationale	Outcome
Pre-provision synthetic snapshot validation	The governance engine evaluates a canonical identity record before the Entra object exists. No Graph API calls are made, no side effects occur, and the engine needed no structural changes to support this mode.	Governance enforcement before identity creation
Post-provision validation against real state	Pre-provision gates confirm the intended state. Post-provision gates confirm what the directory actually contains. Graph calls can partially succeed, so the two checks are both necessary.	End-to-end correctness verified independently
Hold queue as a state machine	Every held record has a named state, a defined transition path, reason codes, and a supervised release mechanism. Invalid transitions are rejected. This is not a discard queue.	Held records are auditable and recoverable
Optimistic concurrency with lock expiry	A lock written at run start expires in ten minutes. This prevents two instances from processing the same event while allowing automatic recovery if the first instance fails.	Safe parallel execution without a coordination service
RBAC via group membership, not direct assignment	RBAC entitlements live on groups, not user objects. Provisioning assigns group membership. RBAC policy changes apply at the group level and take effect without touching individual users.	Maintainable RBAC at scale

10. Technology Stack

The stack reflects the constraints of a Microsoft cloud identity environment. Every component is either a Microsoft-native service or integrates natively with the Microsoft identity platform.

Layer	Technology	Role
Identity Platform	Microsoft Entra ID	The authoritative identity store for Microsoft 365 and Azure. All user objects, group memberships, and directory roles are managed here.
API Integration	Microsoft Graph API	The only interface used for identity operations: user creation, group membership, and directory queries. No alternative API paths.
Orchestration	Azure Functions (Python)	Serverless runtime. HTTP triggers for on-demand provisioning. Timer triggers for scheduled reconciliation.

Layer	Technology	Role
Governance Validation	PowerShell Azure Function	The validation engine runs as a separately deployed function app. The provisioning pipeline calls it over HTTP and receives a structured pass or fail response.
Event Store	Azure Table Storage	Used for event identifier storage, concurrency lock state, and FIFO conflict queue records.
Policy Configuration	JSON (Azure Storage)	Canonical lookup tables, entitlement rules, and governance rule definitions stored as version-controlled files. No redeployment required to update policy.
Privileged Access	PIM via Entra ID P2	Group-based PIM eligibility assignment for just-in-time privileged access. Scoped to Phase 2.
Authentication	Managed Identity	All Graph API and Azure ARM calls use Managed Identity. No credentials are stored or rotated manually.

Microsoft Graph API is the only interface used for identity operations. No secondary API paths, no direct directory writes, no third-party connectors. This keeps all operations on the same auditable API surface and ensures the architecture works across any Entra ID tenant without modification.

11. Validation and Testing

The scenarios below confirm that each control behaves correctly under the condition it was built for. Each test targets a specific failure mode or boundary.

Scenario	Condition	Observed Outcome
Duplicate event submission	Same HR event submitted twice	Second run finds the completed event record and exits. No second provisioning action occurs.
Unknown department value	Payload carries a department value absent from the canonical lookup	Record halted at normalization. Held with rejection reason. Provisioning not reached.
Missing manager association	Payload contains no manager identifier	Pre-provision gate fires on the missing manager rule. Event held for human review.

Scenario	Condition	Observed Outcome
Contractor in management-tier group	Payload targets a management-tier group for a contractor identity	Employment type check fires before any directory write. Provisioning blocked.
RBAC assignment verification	Post-provision state check after a provisioning run	Governance engine confirms actual group memberships match the entitlement model. Discrepancies recorded.
Legacy group absence check	New identity provisioned on a tenant containing legacy groups	Post-provision validation confirms the identity holds no legacy group memberships.
Concurrent function execution	Two instances attempt to process the same identity event	Second instance reads the active lock and exits. No conflicting state is written.
Governance gate rejection	Payload fails one or more pre-provision rules	Event held with rule identifiers and failure reasons. Decision report written.
Stale lock recovery	Function instance fails mid-processing	Lock expires after ten minutes. Event returns to pending and is processed on the next run.

Testing ran against a live Entra ID tenant using the Azure Functions local development runtime. Provisioning, event store, and governance engine calls all executed against real infrastructure. No mock services were used.

12. Operational Boundaries

The boundaries below define what the current implementation covers and what it does not. Stating these explicitly is part of honest architecture documentation.

Boundary	Description
HR data quality	The engine treats HR data as authoritative after normalization. It does not perform upstream validation or correction of source records.
Microsoft identity surfaces only	Provisioning targets Entra ID users, security groups, and group-based RBAC. Application provisioning outside the Microsoft identity platform is out of scope for this phase.
Dynamic group membership	Dynamic membership rules in Entra ID are an optional enhancement and are not required for entitlement resolution. The engine operates correctly without Entra ID P1.

Boundary	Description
PIM requires Entra ID P2	PIM eligibility assignment requires P2 licensing. The engine detects absence at runtime, records a warning, and completes group and RBAC provisioning independently.
Lifecycle Workflows excluded	Entra ID Lifecycle Workflows is excluded by design. The engine is a full replacement for environments requiring explicit policy control and cross-tenant portability.
Queued event release	When a queued event is released after a predecessor completes, it transitions to pending. A new pipeline run, or manual trigger is needed to process it. A timer-triggered release is the planned production approach.
Graph API throttling	Graph API calls use retry logic respecting the Retry-After header. Three attempts are made before a call is marked as failed. Further recovery relies on the event store retry path on the next run.

13. Project Roadmap

The Joiner module is Phase 1 of a four-phase lifecycle system. The event store, hold queue, governance engine interface, and audit layer carry forward into Mover and Leaver without structural changes.

Phase	Module	Scope	Status
0	Foundation	Pipeline contracts, normalization, hold queue state machine, event store, audit system	**Complete**
1	Joiner	Full provisioning pipeline, governance gates, policy-driven entitlement resolution, decision reports	**Complete**
2	PIM Eligibility	Group-based PIM eligible role assignment. Requires Entra ID P2.	**Complete**
3	Mover	Role and department transitions, delta recalculation, revoke-before-add pattern, Retain List support	Designed
4	Leaver	Account disablement, session revocation, full access removal, soft delete with configurable retention	Designed

Phases 0 and 1 need no premium licensing. Entra ID Free covers all core provisioning, RBAC, and governance validation. Phase 2 requires Entra ID P2 for PIM. Dynamic group membership rules, an optional Phase 1 enhancement, require Entra ID P1.

Entra ID Lifecycle Workflows is excluded by design. It targets rapid low-code deployment. This engine targets environments that need explicit policy control, a complete audit trail, and cross-tenant portability. The two approaches serve different requirements.

14. Architecture Summary

The Joiner module provisions new identities into Microsoft Entra ID through a policy-enforced, auditable pipeline. Governance validation runs before identity creation, not after. Access is derived from policy, not from templates or manual selection.

Key properties of the architecture:

- Policy-driven entitlements: every group assignment comes from an externally configurable rule set. Nothing is hardcoded.
- Pre-provision governance enforcement: no Entra ID object is created without clearing a 27-rule validation gate. Failing records are held, not discarded.
- Consistent outcomes: the SHA-256 event identifier guarantees one outcome per event across retry and resubmission.
- Structured decision records: every event produces a JSON report at the time of processing. Compliance evidence is not reconstructed from logs.
- Employment type constraints: Contractors cannot hold management-tier access. This is checked before any directory write.
- Maintainable policy: the rule set, and lookup tables are files in Azure Storage. Policy changes need no code deployment.

The Mover and Leaver phases are designed and ready to build. Mover adds delta calculation and entitlement recalibration. Leaver adds access revocation, session termination, and configurable retention before deletion. The three phases together cover the full identity lifecycle on the Microsoft Azure platform.

Henry Ezihe | Identity and Access Management Engineer | May 2026